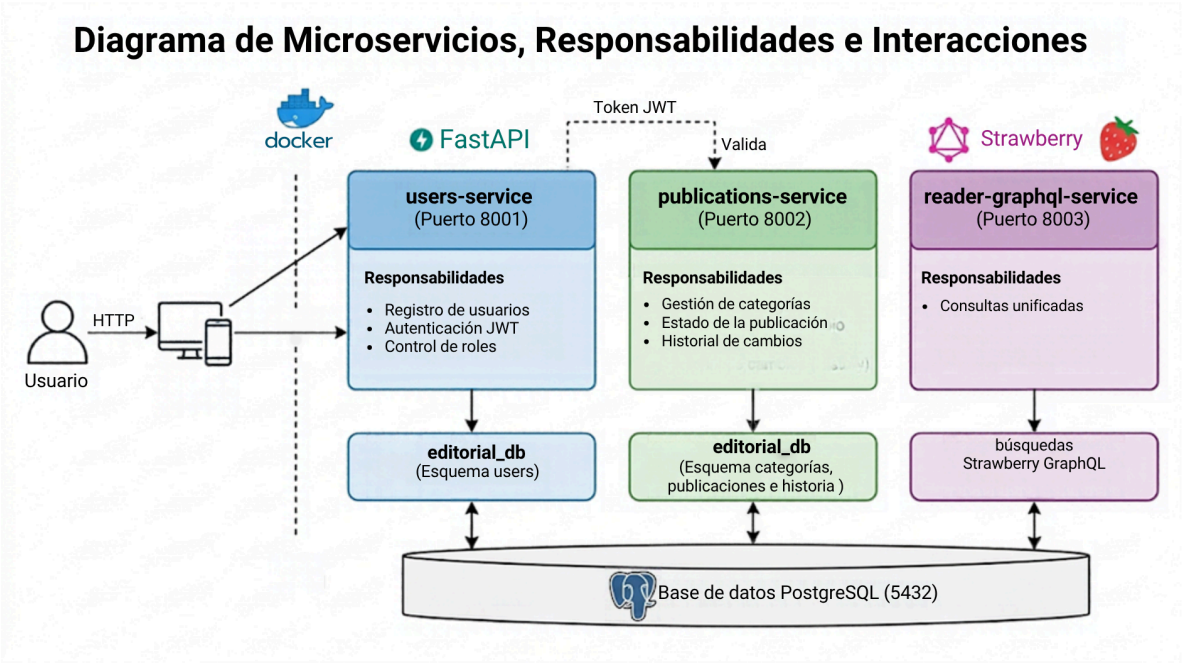


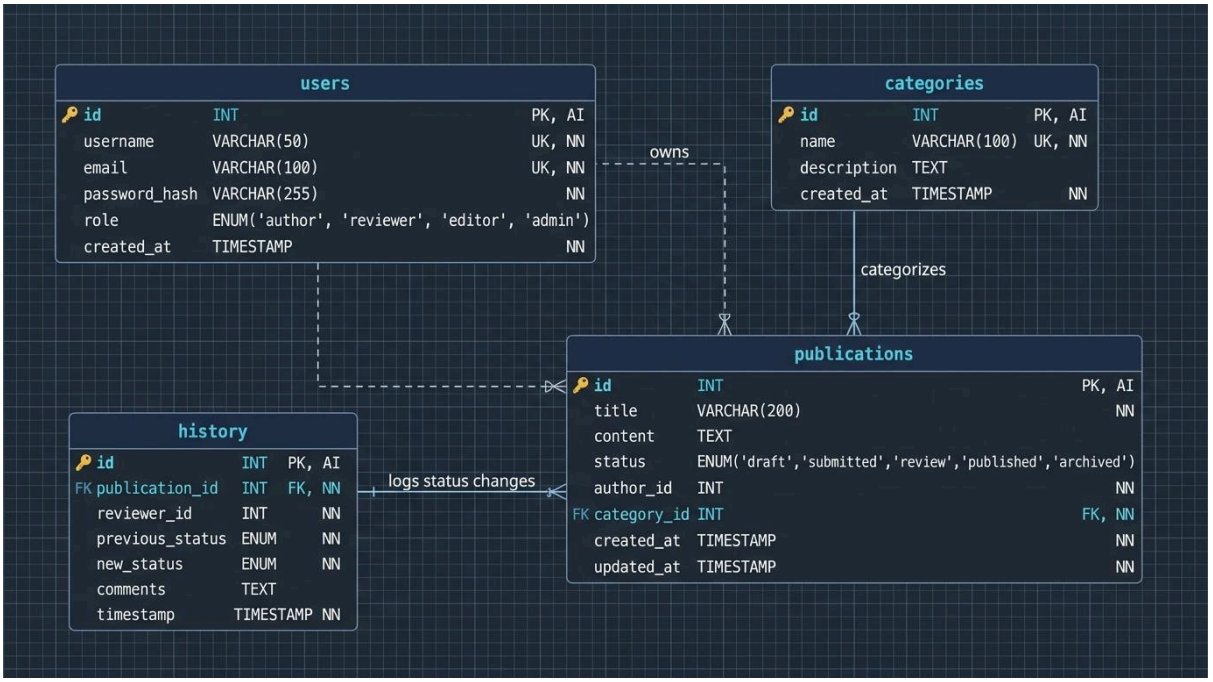
Arquitectura de Microservicios

El sistema se ha descompuesto en tres microservicios para resolver las necesidades del negocio

1. **Servicio de Usuarios y Acceso (`users-service`)**
    - Permite registrar nuevos usuarios, verificar contraseñas y emitir el acceso o Token JWT. Además, define qué rol tiene cada persona
        - **Publicador:** Quien escribe
        - **Aprobador:** Revisa y decide si una publicación sale al público
        - **Lector:** El usuario final que solo lee el contenido
2. **Servicio Editorial (`publications-service`)**
    - Gestiona las categorías y controla el ciclo de vida de los artículos mediante estados:
        - Una publicación nace como `BORRADOR`
        - Pasa a `EN_REVISION` cuando el autor la envía al editor
        - El editor la evalúa y decide si queda `APROBADO` o `RECHAZADO`
        - Finalmente, pasa a `PUBLICADO` para que todos puedan leerla
    - **Historial (Auditoría):** Cada vez que un editor aprueba o rechaza una nota, el sistema anota automáticamente quién lo hizo, a qué hora y con qué comentarios en la tabla `history`
3. **Servicio de Lectura (`reader-graphql-service`)**
    - Es un servicio exclusivo de consulta y lectura. No guarda ni modifica datos; solo busca información en la base de datos usando GraphQL. Permite que el lector pida exactamente lo que necesita ver

¿Por qué se diseñó de esta manera?

- **Independencia y orden:** Si el día de mañana queremos cambiar la forma en que se aprueban las publicaciones, trabajamos únicamente en `publications-service` sin tocar ni poner en riesgo el registro de usuarios ni el sistema de lectura
- **Seguridad:** Cuando un usuario inicia sesión en `users-service`, recibe su token digital (JWT). Cuando quiere crear una publicación, le muestra ese token a `publications-service`. Este último verifica la autenticidad del token, sin tener que llamar a cada segundo a `users-service`. Esto evita tráfico innecesario en la red.
- **Separación entre escribir y consultar:** registrar o editar artículos exige validar permisos por rol, verificar estados y registrar auditorías transaccionales, lo que consume más recursos de procesamiento, la lectura de contenidos presenta un volumen masivo y recurrente que no altera datos, por lo que aislarla en un microservicio de solo lectura con GraphQL garantiza que picos intensos de tráfico de lectores no saturen la memoria



Descripción de las Tablas y Tipos

- **users**
  - **id** (SERIAL, Primary Key): Identificador único del usuario
  - **username** (VARCHAR, Unique, Not Null): Nombre de usuario
  - **email** (VARCHAR, Unique, Not Null): Correo electrónico
  - **password_hash** (VARCHAR, Not Null): Contraseña cifrada con bcrypt
  - **role** (user_role Enum: PUBLICADOR, APROBADOR, LECTOR)
  - **created_at** (TIMESTAMPTZ, Default Current Timestamp)
- **categories**
  - **id** (SERIAL, Primary Key): Identificador de la categoría
  - **name** (VARCHAR, Unique, Indexed, Not Null): Nombre de la categoría
  - **description** (TEXT, Nullable): Descripción editorial opcional

    - **created_at** (TIMESTAMPTZ, Default Current Timestamp): Fecha de registro
- **publications**
    - **id** (SERIAL, Primary Key): Identificador de la publicación
    - **title** (VARCHAR, Indexed, Not Null): Título del artículo o nota
    - **content** (TEXT, Not Null): Cuerpo de la publicación
    - **status** (publication_status Enum: BORRADOR, EN_REVISION, APROBADO, PUBLICADO, RECHAZADO, Default BORRADOR)
    - **author_id** (INTEGER, Indexed, Not Null): ID del usuario creador (asociación lógica con users.id)
    - **category_id** (INTEGER, Foreign Key a categories.id, Not Null): Categoría a la que pertenece
    - **created_at** / **updated_at** (TIMESTAMPTZ): Tiempos de creación y última edición
- **history**
    - **id** (SERIAL, Primary Key): Identificador de la entrada de auditoría
    - **publication_id** (INTEGER, Foreign Key a publications.id, Not Null): Publicación evaluada
    - **reviewer_id** (INTEGER, Not Null): ID del usuario que realizó la acción de aprobación o rechazo (asociación lógica con users.id)
    - **previous_status** (publication_status Enum, Not Null): Estado anterior
    - **new_status** (publication_status Enum, Not Null): Nuevo estado asignado
    - **comments** (TEXT, Nullable): Justificación
    - **timestamp** (TIMESTAMPTZ, Default Current Timestamp): Momento exacto del cambio

Características de la Arquitectura de Datos

- **Relaciones Lógicas (Desacoplamiento de Microservicios):** Los campos author_id y reviewer_id almacenan enteros que corresponden a users.id sin una llave foránea de base de datos directa. Esto mantiene la independencia física de users-service frente a publications-service.

Documentación de las API rest

1. Operaciones de Users Auth Service (users-service)

- **POST /api/users/register (Alta de Usuario):**
    - **Función:** Registra una nueva cuenta de usuario en el sistema dentro de la tabla `User`.
    - **Mecanismo:** Valida que el nombre de usuario y el correo electrónico no existan previamente. Procesa la contraseña mediante la función de derivación de claves Bcrypt, aplicando salting aleatorio, y almacena únicamente el hash resultante. Asigna el rol PUBLICADOR, APROBADOR o LECTOR
    - **Acceso:** Público.
- **POST /api/users/login (Autenticación y Emisión de Token):**

    - **Función:** Valida la identidad del usuario y genera sus credenciales de acceso temporal.
    - **Mecanismo:** Busca el usuario por su nombre de cuenta y ejecuta una comparación entre la contraseña provista y el hash almacenado para mitigar ataques. Si la validación es exitosa, construye y firma criptográficamente un token JWT bajo el algoritmo HS256, empaquetando en sus claims el identificador único (user_id / sub), el nombre de usuario, el rol editorial y la marca temporal de expiración en UTC.
    - **Acceso:** Público.
- **GET /api/users/me (Consulta de Perfil Propio):**
    - **Función:** Permite al usuario en sesión obtener los datos actualizados de su cuenta.
    - **Mecanismo:** Intercepta la cabecera HTTP de autorización Bearer, valida de forma local la firma y vigencia del token JWT mediante `get_current_user`, extrae el identificador del usuario y recupera su información desde el repositorio.
    - **Acceso:** Autenticado (cualquier rol válido).
- **GET /api/users/ (Cuentas Registradas):**
    - **Función:** Retorna el catálogo completo de usuarios registrados en la base de datos.
    - **Mecanismo:** Ejecuta una lectura sobre la tabla de usuarios sin exponer hashes de contraseñas.
    - **Acceso:** Autenticado (requiere token JWT activo).
- **GET /api/users/{user_id} (Consulta de Usuario por Identificador):**
    - **Función:** Recupera la información de perfil de un usuario a partir de su clave.
    - **Mecanismo:** Busca el registro en base de datos; si no existe, interrumpe el flujo emitiendo un error 404.
    - **Acceso:** Público.
- **GET /health (Diagnóstico Operativo):**
    - **Función:** Muestra el estado de infraestructura y orquestadores.
    - **Mecanismo:** Responde inmediatamente confirmando que el proceso ASGI está activo, sin emitir consultas hacia PostgreSQL.
    - **Acceso:** Público.

2. Operaciones de Publications Service (`publications-service`)

- **POST /api/publications/categories (Registro de Categoría Editorial):**
    - **Función:** Da de alta una nueva categoría en la tabla `Category`.
    - **Mecanismo:** Comprueba que no exista una categoría previa con el mismo nombre y persiste la entidad.
    - **Control de Acceso:** Requiere token JWT y perfil restringido a los roles PUBLICADOR o APROBADOR.
- **GET /api/publications/categories (Catálogo de Categorías):**
    - **Función:** Expone el listado completo de categorías disponibles para clasificar artículos.
    - **Mecanismo:** Consulta la tabla de categorías.

    - **Acceso:** Autenticado.
- **POST /api/publications/ (Creación de Publicación):**
    - **Función:** Inicia el ciclo de vida de un contenido digital creando un registro en la tabla `Publication`.
    - **Mecanismo:** Extrae el identificador del autor directamente desde los claims del token JWT, forzando que la publicación quede ligada al usuario autenticado. Valida la existencia física de la categoría especificada y asigna de manera obligatoria el estado inicial BORRADOR.
    - **Control de Acceso:** Restringido estrictamente a usuarios con rol PUBLICADOR.
- **GET /api/publications/ (Listado y Filtrado de Contenidos):**
    - **Función:** Recupera el conjunto de publicaciones registradas.
    - **Mecanismo:** Aplica cláusulas de filtrado por estado editorial (BORRADOR, EN_REVISION, APROBADO, PUBLICADO, RECHAZADO) o por categoría temática.
    - **Acceso:** Público o autenticado.
- **GET /api/publications/{publication_id} (Detalle e Historial Editorial):**
    - **Función:** Obtiene la información de una publicación específica.
    - **Mecanismo:** Carga la entidad mediante su clave primaria y serializa automáticamente los objetos vinculados: la categoría y la colección cronológica completa de cambios de estado auditados en la tabla `History`.
    - **Acceso:** Público o autenticado.
- **PUT /api/publications/{publication_id} (Modificación de Contenido):**
    - **Función:** Permite actualizar el título, el cuerpo del artículo o la categoría asociada.
    - **Reglas de Negocio:**
        - **Propiedad:** Solo el autor original que redactó el artículo tiene autorización para modificarlo; cualquier otro usuario recibe denegación por permisos insuficientes.
        - **Inmutabilidad de Estado:** La edición solo es permitida si el estado actual es BORRADOR o RECHAZADO. Si el artículo se encuentra en revisión o ya fue aprobado, el sistema bloquea la modificación para preservar la integridad editorial.
    - **Control de Acceso:** Requiere token JWT del autor original.
- **PATCH /api/publications/{publication_id}/status (Transición de Estado y Auditoría):**
    - **Función:** Ejecuta el avance o retroceso de un artículo dentro del sistema.
    - **Reglas de Negocio:**
        - **Validación de Estados:** Valida contra una matriz determinista que el paso solicitado sea legal (ej. de BORRADOR a EN_REVISION, o de EN_REVISION a APROBADO/RECHAZADO).
        - **Segregación de Funciones:** Las decisiones de aprobación o rechazo están reservadas con exclusividad al revisor editorial.
        - **Registro Inmutable:** En la misma transacción donde se actualiza el estado, se genera un registro en el historial que preserva el identificador del revisor, el estado anterior, el nuevo estado, las observaciones o motivos del dictamen y la marca temporal exacta.

    - **Control de Acceso:** Restringido estrictamente al rol APROBADOR.
- **GET /health (Diagnóstico Operativo):**
    - **Función:** Sondeo de salud básico del microservicio editorial para balanceadores y monitores de contenedores.
    - **Acceso:** Público.

Esquemas de Graphql

```
scalar DateTime

type HistoryType {
  id: Int!
  publicationId: Int!
  reviewerId: Int!
  previousStatus: String!
  newStatus: String!
  comments: String
  timestamp: DateTime!
}

type CategoryType {
  id: Int!
  name: String!
  description: String
  createdAt: DateTime!
}

type PublicationType {
  id: Int!
  title: String!
  content: String!
  status: String!
  authorId: Int!
  categoryId: Int!
  createdAt: DateTime!
  updatedAt: DateTime!
  category: CategoryType
  history: [HistoryType!]!
}

type Query {
  listPublications(status: String = null, categoryId: Int = null): [PublicationType!]!
  getPublication(publicationId: Int!): PublicationType
  listCategories: [CategoryType!]!
}
```

Justificación de Decisiones de Arquitectura

Descomposición en Microservicios

El sistema abandona el enfoque monolítico tradicional para adoptar tres microservicios: `users-service`, `publications-service` y `reader-graphql-service`.

- **Escalabilidad Asimétrica:** En una editorial, la tasa de lectura supera por a la tasa de redacción o administración. Separar `reader-graphql-service` permite tomar acciones si solo los contenedores de lectura sufren picos de tráfico.
- **Tolerancia a Fallos y Aislamiento de Dominio:** Una degradación en el servicio de autenticación o de gestión de publicaciones no interrumpe la consulta de artículos ya publicados.

Desacoplamiento Lógico de Base de Datos

Aunque los microservicios convergen en una misma instancia de PostgreSQL para optimizar recursos en Docker, se respeta la autonomía de datos:

- **Relaciones Físicas vs. Lógicas:** Las llaves foráneas duras (`FOREIGN KEY`) existen únicamente dentro del contexto de cada microservicio (`history.publication_id -> publications.id` y `publications.category_id -> categories.id`).
- **Límite de Contexto (Bounded Context):** Los campos `author_id` y `reviewer_id` se gestionan como identificadores enteros sin restricciones de integridad referencial física contra la tabla `users`.

2. Aplicación de Principios SOLID

S - Single Responsibility Principle (Principio de Responsabilidad Única)

- **A nivel de Servicios:** Cada microservicio responde a un único actor del sistema: `users-service` administra identidades; `publications-service` gestiona el ciclo editorial; `reader-graphql-service` atiende a los clientes de consumo.
- **A nivel de Código:**
    - Las clases de SQLAlchemy representan exclusivamente el mapeo objeto-relacional (ORM).
    - Los esquemas Pydantic validan y transforman la entrada/salida HTTP.
    - Los controladores de FastAPI orquestan la petición sin mezclar lógica de hashing criptográfico ni construcción directa de consultas SQL dispersas.

O - Open/Closed Principle (Principio de Abierto/Cerrado)

- **Máquina de Estados Extensible:** El flujo editorial implementa transiciones definidas que permiten agregar nuevos estados intermedios (como `EN_CORRECCION` o `ARCHIVADO`) mediante la incorporación de nuevas reglas en la tabla de transición, sin tener que reescribir ni romper la lógica existente de los estados `BORRADOR` o `PUBLICADO`.

- **Esquema GraphQL:** La incorporación de nuevos resolvers o campos complementarios en `PublicationType` no altera ni invalida los contratos de consulta consumidos previamente por clientes antiguos.

L - Liskov Substitution Principle (Principio de Sustitución de Liskov)

- **Esquemas de Entrada y Respuesta:** Las subclases de Pydantic (`PublicationCreate`, `PublicationUpdate`, `PublicationResponse`) extienden de esquemas base garantizando que cualquier método que acepte un modelo base pueda procesar sus especializaciones sin provocar comportamientos anómalos o rupturas de contrato en las APIs.

I - Interface Segregation Principle (Principio de Segregación de Interfaces)

- **GraphQL como Máxima Expresión de Segregación:** En contraste con endpoints REST tradicionales que devuelven estructuras rígidas (provocando *over-fetching* o *under-fetching*), GraphQL permite que cada cliente solicite exactamente la interfaz de datos que necesita. Un cliente móvil solicita únicamente `{ id, title }`, mientras que un panel editorial solicita el grafo extendido con `{ id, title, category { name }, history { newStatus, comments } }`.
- **Endpoints REST Especializados:** Las rutas de actualización de estado (`PATCH /api/publications/{id}/status`) están separadas de la edición de contenido general (`PUT /api/publications/{id}`), evitando forzar al cliente a enviar cargas completas cuando solo requiere alterar una propiedad puntual.

D - Dependency Inversion Principle (Principio de Inversión de Dependencias)

- **Inyección de Dependencias de FastAPI (Depends):** Los controladores de ruta no instancian directamente las sesiones de base de datos ni los servicios de seguridad; dependen de abstracciones inyectadas en tiempo de ejecución:

Stack Tecnológico

- FastAPI
- Strawberry GraphQL
- PostgreSQL
- Pytest + HTTPX

Estrategia y Justificación de Seguridad

Autenticación Desacoplada y Stateless (JWT)

- **Mecanismo:** `users-service` emite un JSON Web Token firmado criptográficamente con algoritmo HMAC-SHA256 (`HS256`).

- **Justificación de Desacoplamiento:** El token contiene en su carga útil (*payload*) la identidad del usuario (`sub`), el correo electrónico y su rol editorial (`role`). Al estar autofirmado, los servicios receptores (`publications-service`) no necesitan realizar una llamada de red de verificación hacia `users-service` por cada petición entrante; simplemente verifican la firma y la fecha de expiración (`exp`) en memoria local utilizando el secreto compartido inyectado vía variables de entorno.

Control de Acceso Basado en Roles (RBAC)

Se implementa una política de privilegios mínimos (*Principle of Least Privilege*):

- **`PUBLICADOR`:** Restringido a crear borradores, gestionar categorías y solicitar la revisión de sus propios artículos. No tiene privilegios para autoaprobarse ni publicar.
- **`APROBADOR`:** Autorizado para auditar, aprobar, rechazar y promover artículos a `PUBLICADO`.
- **`LECTOR`:** Limitado al consumo de información pública a través de la capa de lectura.

Almacenamiento Criptográfico Seguro

- Las contraseñas de usuario se procesan mediante **Bcrypt** con factor de costo configurable (mínimo 12 rondas de salting). La base de datos jamás persiste contraseñas en texto plano, protegiendo los registros contra ataques de diccionario y tablas rainbow en caso de exfiltración de datos.

Seguridad Transaccional e Inmutabilidad de Auditoría

- La máquina de estados editoriales no sobreescribe la traza de decisiones previas. Cada transición hacia `APROBADO`, `RECHAZADO` o `PUBLICADO` ejecuta una transacción atómica que actualiza el estado de la publicación y genera un registro append-only (solo inserción) en la tabla `history`, documentando revisor, estado previo, estado nuevo y observaciones.